IBM Power Porting Triage Agent

Enterprise Workload Qualification & Transitive Migration Sizing for Linux on Power (ppc64le)

Solution Focus: Autonomous Migration Feasibility & Sizing	**Target Architecture:** IBM Power (ppc64le)
Primary Personas: Technical Pre-Sales & Enterprise Architects	**Platforms:** RHEL 8/9, OpenShift (OCP), Ubuntu, SLES

1. Customer Profile & Target Personas

Customer Profile: Global enterprises running high-throughput transactional, time-series, and enterprise AI workloads (financial services, insurance, telecom, retail, and healthcare) evaluating platform modernization from commodity x86 infrastructure to **IBM Power (ppc64le)** on RHEL and Red Hat OpenShift Container Platform (OCP).

- **IBM Technical Pre-Sales Specialists & Solution Architects:** Frontline technical sellers who must rapidly qualify migration feasibility, size porting timelines, and build customer confidence during competitive evaluations.
- **Enterprise Architecture Decision-Makers (CIO / CTO / VP Infrastructure):** Executive stakeholders requiring transparent risk qualification, timeline predictability, and platform ROI justification before approving modernization contracts.
- **IBM Power Porting Lab & Ecosystem Engineering Squads:** Specialized engineering squads responsible for compiling, tuning, and certifying non-native packages, who need structured, production-grade technical handoffs.

2. Business Problem & Pre-Sales Friction

Modernizing to IBM Power delivers transformative hardware throughput (**industry-leading memory bandwidth**, **SMT8 core threading**, and integrated **Matrix Math Accelerators / MMA**). However, winning executive platform commitment hinges on answering a fundamental question during pre-sales: *'Will our software stack run on IBM Power, and what is the exact engineering timeline required to address any gaps?'* Today, answering this question introduces severe friction into the enterprise sales cycle:

- **Unstandardized Ingestion Formats:** Customers provide dependency specifications in fragmented formats: CycloneDX/SPDX SBOMs, multi-stage Dockerfiles, Python requirements, Compose/Kubernetes YAMLs, or meeting transcripts.
- **Fragmented Registries & Version Drift:** Validating package availability requires tedious searches across RHEL BaseOS, EPEL 8/9, Quay.io, Docker Hub, PyPI, and community channels. Generic queries often mask version drift where the client's specific tag lacks a ppc64le binary.
- **The Unseen 'Dependency Iceberg':** When a C/C++ library lacks an official binary, standard triage fails to anticipate transitive build requirements (e.g. jemalloc needing 64KB page size tuning) or hardcoded x86 SIMD intrinsics (AVX2/AVX-512), causing delivery overruns.
- **Subjective Timeline Estimation:** Heuristic guesswork produces 2x–5x variances that erode trust between sales teams and client executive stakeholders.

3. Quantified Operational Impact

Operational Dimension	Traditional Manual Process	Autonomous Triage Agent Solution
Qualification Velocity	5 to 10 Business Days across manual catalog lookups and emails	**< 30 Seconds automated end-to-end qualification and analysis**
Engineering Bandwidth Waste	~40% of senior porting lab capacity spent on routine initial lookups	**Zero wasted hours on routine triage; lab only receives scoped specs**
Pre-Sales Deal Velocity	Prolonged sales cycles; customer hesitation due to unknown porting risks	**Immediate client alignment via traffic-light scoring & transparent derivation**
Effort Sizing Defensibility	Subjective guesswork prone to significant variance from hidden dependencies	**Mathematical 6-Stage WBS + Multi-Tier Complexity & Debugging Room**
Multi-Tier Deliverable Scoping	Ambiguity between Container, Source Build, and Build Script needs	**Automated deliverable alignment (Containers, Builds, Build Recipes)**
Executive Proposal Deliverables	Days spent assembling custom slide decks and proposal memos	**1-Click Executive PDF Memo with provenance & JIRA CSV backlog**

4. Solution Architecture & Ecosystem Intelligence Pipeline

The **IBM Power Porting Triage Agent** seamlessly bridges deterministic ecosystem registry connectors, a persistent mounted knowledge cache, dual LLM reasoning engines (IBM Bob Shell & Google Gemini), and an enterprise lifecycle sizing engine to provide real-time feasibility analysis and defensible delivery estimates:

- **Streamlined Dual-Input Ingestion:** Ingests CycloneDX/SPDX SBOMs, multi-stage Dockerfiles (FROM ... AS builder), Python requirements, Docker Compose, Kubernetes manifests, raw client text, and direct **Git repository & documentation URLs** (e.g., https://github.com/openai/whisper or https://rocksdb.org).
- **Dual LLM Reasoning Engine (IBM Bob Shell & Gemini):** Features first-class native support for **IBM Bob Shell (ibm_bob)** headless execution alongside Google Gemini. Auto-detects local CLI binaries and keys with dynamic fallback and neutral, provider-agnostic telemetry in the live activity feed.
- **Persistent Mounted Package Knowledge Cache:** Maintains a filesystem-mounted cache (backend/data/packages_lookup.json at /app/backend/data) that persists and dynamically refreshes analyzed packages and versions, eliminating repetitive LLM and registry lookups.
- **Multi-Tier Package Complexity & Dedicated Debugging Room:** Replaces flat 1 PD assumptions with dynamic complexity tiers (Systems/Storage: 5.5–8.5d, Vector Math: 8.5–9d, Native C/C++: 3.5–4d, Rust/Go: 3d). Explicitly allocates **1.0–1.5 PD** for compiler flag adaptation, 64KB page-size alignment, and test suite debugging.
- **IBM DevPi Wheels & ICR Multi-Arch Containers:** Directly matches optimized ppc64le wheels (PyTorch, Numba, NumPy, Tiktoken) in IBM DevPi and enterprise multi-arch images in IBM Cloud Registry (ICR) and ppc64le/build-scripts.
- **Multi-Source Prober with Version-Aware Intelligence:** Queries Red Hat Ecosystem Catalog, Fedora Koji / EPEL 8/9 XML-RPC APIs, Quay.io, Docker Hub v2 API, and PyPI with real-time version mismatch and upgrade/downgrade advice.
- **Deliverable-Aware Alignment & Matching:** Evaluates packages against **Container Image**, **Source Build**, or **Build Script / Recipe**, calculating adaptation multipliers when artifacts require format bridging.
- **Deep Transitive Build Dependency Scoping:** Recursively inspects build requirements (BuildRequires, submodules, header libraries) for non-native components to uncover hidden dependencies before compilation starts (e.g. scoping jemalloc-ppc64le for RocksDB).
- **Hardware Architecture Sensitivity Auditing:** Audits source code for CPU friction points: flags x86 SIMD (AVX2/AVX-512) intrinsics (recommending SIMDe to Power VSX translations), 64KB page size sensitivities (jemalloc tuning), and inline assembly (x86 pause mapped to Power or 27,27,27 yield).
- **Upstream CI Matrix & Architecture Posture Scanning:** Audits upstream Git repositories for existing Power markers (#ifdef __powerpc__) and CI matrices for active ppc64le runners, applying an evidence-based effort discount (0.7x) for Power-aware codebases.
- **Token & Cost Accounting with Collapsible Stage Breakdown:** Real-time tracking of input/output tokens and cost in USD. The metric tile on the Executive Scorecard features an interactive collapsible drawer detailing stage-by-stage token and cost contributions.

Streamlined Ingestion SBOM, Dockerfile, Git / Doc URLs, Transcripts	**Multi-Source & Cache** Persistent Lookup, DevPi, ICR, build-scripts, Koji	**Complexity & Arch Audit** SIMD Brackets, 64K Pages, Toolchains, Debug Room	**Executive Synthesis** 6-Stage WBS, Dual LLM (Bob/Gemini), 1-Click PDF

5. Transparent Sizing Methodology & Lifecycle WBS

To replace subjective estimation with enterprise delivery rigor and eliminate legacy Fibonacci sizing, the agent computes porting effort across an auditable **6-Stage Porting Lifecycle Work Breakdown Structure (WBS)** with an explicit 20–25% contingency buffer:

6-Stage Lifecycle WBS: Subtotal Effort = Prep Env & Toolchain + Readiness Audit + Build from Source + CI Pipeline + Bug Fix & Stabilization
Risk Contingency Buffer: +20% (Standard) or +25% (Architecture Friction / Deep Transitive Iceberg, min 1.0 PD)
Consistency Guarantee: Scorecard Committed Effort = ceil(Subtotal + Buffer) >= Sum of Package Rows (never flat 1 PD).

Sizing Parameters & Multiplier Matrix

Sizing Dimension	Classification Brackets / Tiers	Multiplier / Scope	Engineering Scope & Rationale
Package Complexity Tiers	Systems/Storage Engine Custom Vector/DSP Math Native C/C++ Library Rust / Go / Python C-Ext Baseline Portable Floor	**5.5 – 8.5 PD** **8.5 – 9.0 PD** **3.5 – 4.0 PD** **3.0 PD** **3.0 PD**	Evaluates ecosystem, toolchains, memory barriers, 64KB page kernels, with **1.0–1.5 PD allocated for test suite execution & debugging room**.
SIMD Instruction Volume	0 – 50 instructions 51 – 200 instructions 201 – 500 instructions > 500 instructions	**1.0x** **1.5x** **2.5x** **4.0x**	Reflects total vector instruction density requiring manual inspection, inline substitution, and micro-benchmarking.
Porting Complexity Tier	DIRECT SIMDE_COMPATIBLE PARTIAL_REWRITE FULL_REDESIGN	**1.0x** **1.5x** **2.5x** **4.0x**	From 1:1 intrinsic substitution, to SIMDe header mapping, loop refactoring, up to full AVX-512 redesign to Power MMA.
Repository CI Posture	Power CI present (ppc64le runner) Neutral / Mixed markers Hardcoded x86 / No Power CI	**0.7x (Discount)** **1.0x (Baseline)** **1.3x (Friction)**	Accounts for upstream willingness and established continuous integration to validate and maintain Power patches.

Worked Enterprise Sizing Derivations

Target Component / Stack	Component Analysis & Findings	Step-by-Step Derivation Formula	Committed Sizing
OpenAI Whisper (Speech-to-Text Pipeline)	• DevPi: Torch, Numba, NumPy, Tiktoken native • Build: Whisper core package (3.0d) • Debug Room: 1.0d test harness & linker triage	**Prep:** 1.0d | **Readiness:** 0.5d | **Build:** 3.0d **CI:** 1.0d | **Bug Fix & Test:** 1.5d **Subtotal:** 7.0d | **Contingency (20%):** 1.4d	**7 PD** **(Minor Effort)**
Storage Engine Stack (RocksDB 8.6 + simdjson 3.6)	• RocksDB (6.0d): 64KB-page jemalloc + WAL audit • simdjson (10.0d): 320 AVX2 instructions • Test: dbbench + cxxopts validation suites	**Prep:** 1.5d | **Readiness:** 1.5d | **Build:** 14.5d **CI:** 2.0d | **Bug Fix & Test:** 3.0d **Subtotal:** 22.5d | **Contingency (25%):** 5.6d	**26 PD** **(Moderate Effort)**

6. Demonstrated Workload Scenarios & Validation

The solution was validated across six representative enterprise migration workloads:

Scenario / Workload	Stack Components	Triage Findings & Multi-Source Matches	Score & Sizing
Enterprise Cloud Microservices	Nginx, Redis, PostgreSQL, Strimzi Kafka, Node.js	All components have verified native ppc64le container images in Docker Hub, Quay.io, and Red Hat Catalog. Defaulted to container deliverable.	**Turnkey (100%)** **0 Person-Days** (Direct binary)
OpenAI Whisper Pipeline (Git URL Ingestion)	Whisper, PyTorch 2.0, Numba, NumPy, Tiktoken, Triton	PyTorch, Numba, NumPy, and Tiktoken matched in IBM DevPi Wheels. Whisper core build scoped with native compilation & debug room (3 PD); lifecycle WBS applied.	**Minor Effort** **7 Person-Days** (Lifecycle WBS)
AI / ML Vector Pipeline	PyTorch, NumPy, Pandas, Scipy, Custom C++ DSP	PyTorch routed to IBM Open-CE with Power MMA acceleration; custom DSP kernel scoped with SIMDe AVX2->VSX remediation and precision validation (8.5 PD).	**Minor Effort** **9 Person-Days** (Lifecycle WBS)
High-Throughput Storage Engine	RocksDB 8.6, simdjson 3.6, Redis, zlib	Deep dependency iceberg scoped: RocksDB (6 PD) with 64KB-page jemalloc, simdjson (10 PD) with VSX vector scaling, full regression suites & debug room.	**Moderate Effort** **26 Person-Days** (Lifecycle WBS)
High-Performance Storage (SPDK)	SPDK 23.05, DPDK, libaio, numactl	SPDK native C/Meson storage engine scoped: 64KB page kernel hugepage buffers, DPDK integration, Power memory barriers, and DMA debugging room (8 PD).	**Moderate Effort** **16 Person-Days** (Lifecycle WBS)
Legacy Financial Analytics	Intel MKL, Python 3.9, Proprietary Risk Calc	Proprietary closed-source x86 blocker flagged; constructive recommendation to substitute with **IBM ESSL** or OpenBLAS drop-in replacement.	**Alternative Req.** **Drop-in ESSL** (OpenBLAS)

7. Strategic Value to IBM & Enterprise Clients

The IBM Power Porting Triage Agent accelerates the enterprise modernization pipeline and eliminates delivery friction:

1. **Compresses Sales Cycles:** Reduces pre-sales qualification turnaround from 5–10 business days to under 30 seconds, arming client teams with authoritative migration roadmaps within seconds of first contact.
2. **Optimizes Engineering Capacity:** Shields the IBM Power Porting Lab from routine package lookups, ensuring senior specialists focus exclusively on scoped, high-value unported libraries.
3. **Establishes Unquestioned Sizing Credibility:** Replaces heuristic guesswork with a transparent 6-stage lifecycle WBS, multi-tier complexity analysis with debugging room, and an interactive derivation drawer.
4. **Minimizes Operational Cost & Latency:** Persistent mounted lookup cache avoids redundant LLM token expenditures and registry latency, while supporting dual LLM engines (IBM Bob Shell and Google Gemini).
5. **Actionable Developer Deliverables:** Generates one-click executive proposals alongside structured JIRA-ready CSV engineering backlogs ready for immediate sprint execution.

Executive Deployment Status: The IBM Power Porting Triage Agent is fully operational and containerized via Podman and Docker Compose. It features native integration with seven ecosystem registries, dual LLM engine execution (IBM Bob Shell & Google Gemini), persistent mounted lookup caching, and delivers compliant IBM Carbon Design System (g100) interfaces ready for client demonstrations and automated API deployment.